

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Ingeniería de Software

Dra. Archundia Sierra Etelvina

Primavera 2023

Equipo #2

Entregable del capítulo I

Castro Salgado María Fernanda	202031397
Díaz Márquez Tadeo Rafael	202134032
Juárez Gonzáles Berenice	202042438
Meléndez Covarrubias Jesús Iván	202086619
Vázquez Orihuela Oscar	202077134

Contenido

Introducción	3
Planteamiento del problema	4
Metodología de trabajo.....	5
Objetivos generales	7
Objetivo específico	7
Justificación.....	8
Alcances.....	10
Limitaciones.....	11
Recursos humanos, de software y hardware	12
Agenda de trabajo	14
Marcos teóricos	15
Uso del Lenguaje de programación	30
Confiableidad y Seguridad	34
Conclusiones	43
Bibliografía	45
Anexo	48

Introducción

Partiendo de la demanda sobre la utilización de la Red Urbana de Transporte Articulado en la ciudad de Puebla nos topamos con varias problemáticas interesantes de resolver, la que se presenta en el documento que usted sostiene es la siguiente “¿Existe una manera más sencilla de recargar mi tarjeta para abordar el transporte público?”. En este contexto, nos hemos propuesto explorar una solución a través de la ingeniería de software y la creación de una aplicación móvil, denominada RUTAmobile.

Esta aplicación permitiría a los usuarios del RUTA mantener el saldo necesario en su tarjeta, evitando así el estrés e incomodidad de las filas en las estaciones concurridas. De esta manera, tanto estudiantes, trabajadores como amas de casa y demás personas, podrán abordar el transporte público de manera más eficiente y rápida.

En las siguientes páginas encontraremos la estructura para concretar dicho proyecto con la finalidad de mejorar la calidad y servicio del transporte público urbano.

Planteamiento del problema

La Red Articulada de Transporte Urbano requiere de mejorar la forma en que se realizan las recargas a las tarjetas para usar dicho transporte. Es incomodo e ineficiente que haya solo dos máquinas para realizar el abono y que, en muchas ocasiones, solo funcione una, hablamos de largas filas de espera (Especialmente en estaciones como Ciudad Universitaria, Crystal, Paseo Bravo, etc.) que no permiten recargas completamente eficientes (el uso de los centavos o cantidades exactas resulta cómodo para los usuarios que miden sus finanzas).

Una encuesta realizada a usuarios del RUTA muestra que el 80% de las personas se ha topado con una fila considerable de gente para poder ingresar dinero a su tarjeta en más de cuatro ocasiones por semana. Este problema no solo genera inconformidad, sino que también aumenta el tiempo de viaje, la incomodidad y el estrés, especialmente para aquellos usuarios que tienen que llegar a un lugar determinado a una hora específica.

En este contexto, la aplicación móvil RUTAmobile se presenta como una solución eficiente y eficaz a estas problemáticas, permitiendo a los usuarios mantener el saldo necesario en su tarjeta de transporte y evitar las largas filas en las estaciones de servicio.

Metodología de trabajo

Para la realización de este proyecto consideramos factores tales como la capacidad evolutiva del equipo y de la aplicación RUTAmobile. Sabemos de las necesidades que va a requerir nuestro proyecto, en esta ocasión tomaremos a la metodología clásica de Cascada como punto de partida, pues se adapta mejor a lo que estamos enfocados como equipo.

Presentaremos nuestro proyecto ante una entidad gubernamental con la finalidad de que acepten implementarlo tal y como se practica en otras partes del país, mejorando así significativamente el uso del transporte público.

1. Fase de análisis: Observaremos los requerimientos del usuario y del negocio, así mismo es necesario comprender las necesidad y expectativas que cumpliremos para el usuario y la empresa, reconocimiento de requisitos funcionales, no funcionales, objetivos y demás, partiendo de encuestas realizadas por el equipo de trabajo, análisis de casos de uso, y resultados obtenidos por otras entidades que usamos como inspiración.
2. Fase de diseño: con base a los requisitos identificados, componentes del sistema, arquitecturas, diseño de la base de datos, etc., procedemos a crear un primer entregable teórico sobre como pretendemos llegar a una solución física. Este incluirá el uso de diagramas UML y prototipos eficaces.

3. Fase de desarrollo: Iniciamos con la creación de los entregables físicos en fase primeriza, con los requerimientos iniciales considerados, implicando codificación del software, integración de componentes, prueba de unidad, además de la retroalimentación por cada entregable.
4. Fase de verificación: Analizamos si la relación entre lo que propusimos y los entregables es correcta e incluso llegó (o no) a superar las expectativas planteadas, encontramos pruebas de integración, de sistema y de aceptación en menor escala para la corrección posterior de aquello que resulte ineficiente. Entre ellas despliegues a sectores de comprobación de calidad.
5. Fase de implementación: Obtenido el resultado final que cumpla con los requisitos necesarios para ser confiable, eficaz y eficiente, RUTAmobile se presenta a un entorno de producción y puesta en marcha para llegar a los usuarios finales y que estos puedan hacer uso de la aplicación. Dando a conocer a los usuarios mediante campañas de publicidad que ya está disponible la aplicación, al alcance de los cientos de personas que todos los días utilizan la Red Urbana de Transporte Articulado.
6. Fase de mantenimiento: Siempre con la vista en la mejora, hablamos de que es necesario mantenernos alerta sobre los requerimientos de nuestra aplicación, puesto que aun considerando que hablamos del largo plazo para las mejoras de RUTAmobile, somos conscientes de que la tecnología debe estar en constante cambio y mejora, esto incluirá conocer la opinión de los usuarios finales mediante periódicas encuestas que planeamos sean enviadas desde la misma aplicación.

Objetivos generales

- Reducción de los tiempos de espera dentro de las estaciones (especialmente las más concurridas de la ciudad) facilitando el acceso a la zona de abordaje.
- Mejor manejo del dinero recabado por el gobierno estatal a partir del uso del transporte público ya que una parte de ellos se controlaría de forma digital.
- Actualización continua de los elementos como descuentos, promociones y demás que contribuyan a la permanencia del RUTA como principal transporte público.
- Autonomía financiera del usuario al decidir cuanto dinero recargar en su tarjeta para el uso del RUTA.

Objetivo específico

- Implementar una aplicación móvil cuya finalidad sea la de recargar las tarjetas para ingresar a las estaciones del RUTA, evitando así las filas de espera en las máquinas de recarga, mejorando la experiencia de los usuarios del transporte público, reduciendo costos de mantenimiento en máquinas y ahondando en el mundo digital como parte de la cotidianidad de quienes utilizan la Red Urbana de Transporte Articulado.

Justificación

El presente proyecto hace alusión al Gran Objetivo número 9 de la UNESCO en donde hallamos que la infraestructura dentro de los países debe ser considerada un factor muy importante para el desarrollo de las personas que viven en ellos, pilar fundamental sobre el cual construir ciudades sostenibles y de desarrollo acelerado.

Tomamos a consideración distintos factores para creer en este proyecto, tales como:

- Comodidad y conveniencia de los usuarios: sin filas, rápida entrada al transporte público.
- Reducción de costos y aumento de la eficiencia: disminución de las maquinas que realizan las recargas para las tarjetas de RUTA.
- Mayor control y seguridad: las recargas se harían desde la comodidad del hogar para salir sin la preocupación de no llevar dinero para un pasaje.
- Posibilidad de descuentos: implementación de las tarjetas para estudiantes (con validación previa) y también para los adultos de la tercera edad o en situación convaleciente.

Siendo así, los usuarios del transporte público tendrían una satisfacción mayor al tratarse de una iniciativa que favorece a los mismo, evitamos largas filas de espera (especialmente en las estaciones más transitadas), manejo del tiempo de forma eficiente e incluso un mayor control del dinero generado e ingresado al gobierno federal (explicado más adelante pero que, a grandes

rasgos, hace referencia hacia donde llegara el dinero recargado).

Es con este panorama que damos pie a la creación del proyecto RUTAmobile, una aplicación de fácil uso que permitirá alcanzar los factores mencionados anteriormente. Consideramos así un impacto positivo entre los usuarios finales, es decir alrededor del 80% a 90% de satisfacción que mediremos a través de encuestas de respuesta voluntaria.

Alcances

- Desarrollo de una aplicación móvil para la recarga de tarjetas del sistema de transporte público RUTA.
- Implementación de una base de datos para almacenar la información de los usuarios y transacciones.
- Integración de métodos de pago seguros, como tarjeta de crédito o débito y transferencias interbancarias.
- Diseño de una interfaz amigable y fácil de usar para los usuarios.
- La aplicación podrá reportar fallas ocurridas en las estaciones (Es decir, enviar a los usuarios reportes sobre un atasco y, por ende, atraso, de la llegada del transporte).

Limitaciones

- La aplicación solo estará disponible para dispositivos móviles con sistemas operativos iOS y Android.
- La aplicación solo permitirá recargas de tarjetas RUTA, no se podrán realizar otras transacciones.
- La aplicación no podrá utilizarse en lugares donde no haya señal de internet o en lugares con mala conexión.
- La aplicación no funcionara para que el usuario reporte fallas de las maquinas existentes para las recargas del RUTA.

Recursos humanos, de software y hardware

Gerente de proyecto: responsable de la planificación, coordinación y ejecución del proyecto en su totalidad, así como de la comunicación con los demás miembros del equipo y con los stakeholders externos.

Desarrolladores de software: encargados de la programación y el desarrollo de la aplicación móvil, en estrecha colaboración con los diseñadores y el equipo de testing.

Diseñadores: responsables de la creación y diseño de la interfaz de usuario, la imagen corporativa y la propuesta de valor del producto.

Especialista en testing: encargado de realizar pruebas exhaustivas de la aplicación, detectar y reportar errores para que el equipo de desarrollo pueda corregirlos.

Analista de datos: encargado de recopilar y analizar datos de uso y de comportamiento de los usuarios para mejorar la experiencia de uso y la eficacia de la aplicación.

Asesor legal: responsable de garantizar el cumplimiento de las leyes y regulaciones aplicables, y de proporcionar asesoramiento jurídico en caso de que sea necesario.

Equipo de soporte técnico: encargado de brindar asistencia técnica y solucionar problemas relacionados con la aplicación para los usuarios finales.

Software de desarrollo: como IDEs (entornos de desarrollo integrados), lenguajes de programación, herramientas

de control de versiones, software de gestión de proyectos, entre otros.

Servidores: para alojar la base de datos y la aplicación en la nube.

Dispositivos móviles: para probar y verificar la aplicación en diferentes sistemas operativos y dispositivos.

Agenda de trabajo

The image displays two screenshots of the BUAP Planner application, showing a project schedule for 'RUTAmobile'.

Top Screenshot:

- Header:** BUAP Planner. Navigation tabs: Cuadrícula, Panel, Gráficos, Programación. Filters: Miembros, Filtro (0), Agrupar por Depósito.
- Left Sidebar:** Menu with options: Nuevo plan, Sitio central, Asignado a mí, Anclado, RUTAmobile, Todos.
- Main Content:** Project phases and tasks:
 - Fase de análisis:** Tasks include 'G. de proyecto' (03/03) and 'Diseño de aplicación' (24/03).
 - Fase de diseño:** Tasks include 'G. de proyecto' (24/03) and 'Diseño de aplicación' (13/03).
 - Fase de desarrollo:** Tasks include 'G. de proyecto' (20/05) and 'Programación de la...' (09/04).
 - Fase de v...** (partially visible).

Bottom Screenshot:

- Header:** BUAP Planner. Navigation tabs: Cuadrícula, Panel, Gráficos, Programación. Filters: Miembros, Filtro (0), Agrupar por Depósito.
- Left Sidebar:** Menu with options: Nuevo plan, Sitio central, Asignado a mí, Anclado, RUTAmobile, Todos.
- Main Content:** Project phases and tasks:
 - Fase de verificación:** Tasks include 'G. de proyecto' (14/03) and 'Diseño de aplicación' (14/03).
 - Fase de implementación:** Tasks include 'Analista de BDB' (10/05) and 'Marketing' (04/05).
 - Fase de mantenimiento:** Tasks include 'Marketing' (11/03) and 'G. de proyecto' (11/05).
 - Agregar:** (partially visible).

Marcos teóricos

La ingeniería de software es una disciplina que se encarga de aplicar principios y técnicas de ingeniería al desarrollo de software. La ingeniería de software abarca todo el ciclo de vida del software, desde la concepción hasta el mantenimiento, y tiene como objetivo producir software de alta calidad que satisfaga las necesidades del usuario y se entregue en tiempo y forma.[1]

Una de las metodologías más utilizadas en la ingeniería de software es la metodología cascada, también conocida como modelo en cascada. Esta metodología se basa en una secuencia de fases que se ejecutan de manera secuencial, en la que cada fase debe completarse antes de pasar a la siguiente. Las fases principales de la metodología cascada son:

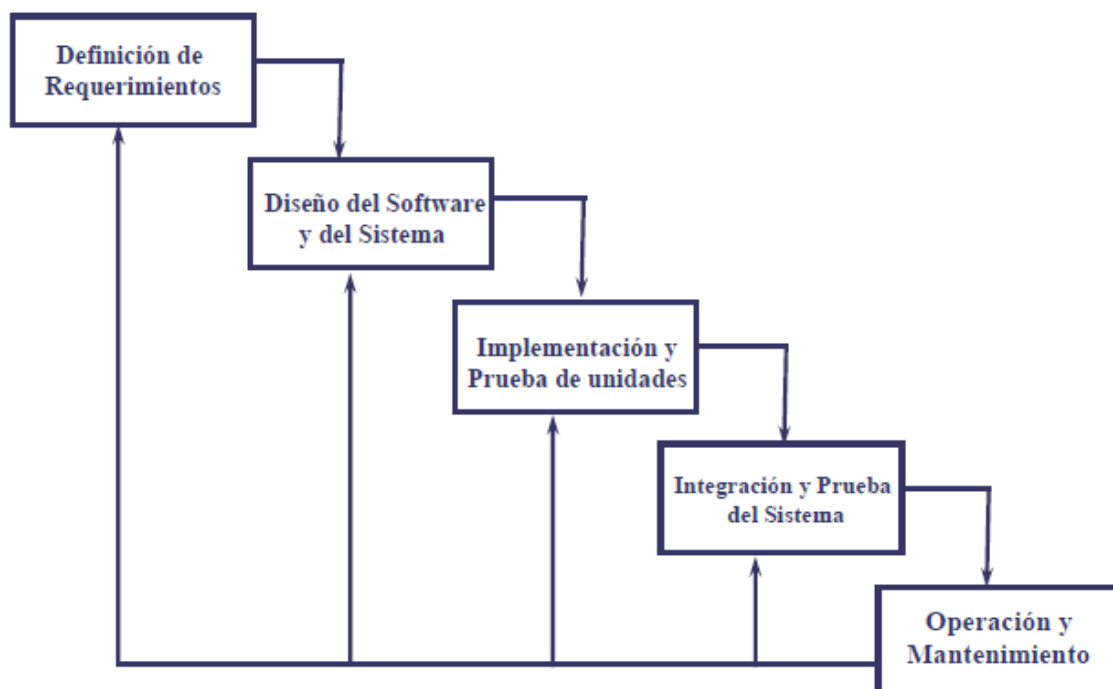
- **Análisis de requisitos:** Esta fase se enfoca en la identificación y especificación de los requisitos del software. Es fundamental establecer una comunicación clara con el cliente para entender sus necesidades y expectativas.
- **Diseño:** En esta fase se diseña la arquitectura del software y se elaboran los diseños detallados de cada uno de los componentes del sistema. Se establecen los estándares y los procedimientos de programación para asegurar que el software se pueda desarrollar de manera uniforme y coherente.
- **Implementación:** En esta fase se lleva a cabo la codificación y el desarrollo del software. Se sigue el diseño establecido en la fase anterior para crear el código fuente del software.
- **Pruebas:** En esta fase se realizan pruebas para verificar que el software cumple con los requisitos especificados. Se

prueban diferentes aspectos del software, como la funcionalidad, la seguridad, la eficiencia y la usabilidad.

- **Mantenimiento:** En esta fase se realizan las correcciones y mejoras necesarias para asegurar que el software siga funcionando de manera adecuada.

La metodología cascada se utiliza principalmente en proyectos grandes y complejos, ya que permite una planificación clara y precisa de cada una de las fases del proyecto. Sin embargo, esta metodología puede presentar algunas desventajas, como una menor flexibilidad en caso de que sea necesario hacer cambios en el proyecto una vez iniciado.

En conclusión, la metodología cascada es una de las metodologías más utilizadas en la ingeniería de software. Esta metodología se basa en una secuencia de fases que se ejecutan de manera secuencial y permite una planificación clara y precisa de cada una de las fases del proyecto. Aunque puede presentar algunas desventajas, la metodología cascada es una herramienta útil para el desarrollo de proyectos grandes y complejos.



Los sistemas de transporte son un componente esencial en la mayoría de las ciudades del mundo, y el pago de los viajes es una parte importante de este proceso. En la actualidad, muchas ciudades han adoptado sistemas de pago electrónicos, que permiten a los usuarios utilizar una tarjeta para pagar su viaje en lugar de tener que pagar en efectivo.

Los proyectos de transporte pagado con una tarjeta se refieren a la implementación de un sistema de pago electrónico en un sistema de transporte público. Estos sistemas de pago electrónico son conocidos como tarjetas inteligentes o tarjetas de transporte público. Estas tarjetas pueden ser recargables y permiten a los usuarios pagar sus viajes con una simple validación en un lector de tarjetas.

Los sistemas de transporte pagado con una tarjeta presentan una serie de beneficios para los usuarios, entre los que se incluyen:

- Mayor comodidad: Los usuarios no tienen que preocuparse por llevar dinero en efectivo para pagar su viaje. En lugar de ello, pueden utilizar su tarjeta de transporte público para pagar.
- Mayor seguridad: Al no tener que llevar dinero en efectivo, los usuarios están menos expuestos al robo y la pérdida de su dinero.
- Mejora de la eficiencia del sistema de transporte: Los sistemas de transporte pagado con una tarjeta pueden mejorar la eficiencia del sistema al reducir los tiempos de espera en las paradas de transporte público.

- Recopilación de datos: Los sistemas de transporte pagado con una tarjeta pueden recopilar datos sobre el uso del transporte público, lo que permite a las autoridades planificar y optimizar el sistema de transporte de manera más efectiva.

La implementación de un sistema de transporte pagado con una tarjeta requiere una planificación cuidadosa y una inversión significativa en infraestructura. Es necesario instalar lectores de tarjetas en las paradas de transporte público y en los vehículos de transporte, así como establecer sistemas de recarga de tarjetas y de gestión de pagos. Además, es necesario capacitar a los usuarios en el uso de las tarjetas de transporte público.

Ahor bien, el uso de diagramas de lectura relativamente sencilla, con un punto clave para la elaboración de un proyecto, puesto que nos va a permitir visualizar de una forma eficaz y entendible el desarrollo y requerimientos de este.

Hay varios tipos de diagramas UML que se utilizan comúnmente en la ingeniería de software, cada uno con un propósito y una funcionalidad específicos. Algunos de los diagramas UML más comunes incluyen:

- Diagramas de casos de uso: se utilizan para representar los diferentes escenarios en los que los usuarios interactúan con el software. Los casos de uso ayudan a los desarrolladores a identificar los requisitos del sistema y a comprender cómo los usuarios interactúan con él.

- Diagramas de clases: se utilizan para representar la estructura del sistema y las relaciones entre las diferentes clases. Este tipo de diagrama puede ser muy útil para identificar las clases necesarias para implementar un sistema y para comprender cómo se relacionan entre sí.
- Diagramas de secuencia: se utilizan para representar la interacción entre diferentes objetos en el sistema. Este tipo de diagrama puede ser muy útil para comprender cómo se lleva a cabo una determinada acción en el sistema.
- Diagramas de actividad: se utilizan para representar el flujo de trabajo en el sistema. Este tipo de diagrama puede ser muy útil para comprender cómo se llevan a cabo diferentes procesos dentro del sistema.

En general, los diagramas UML son una herramienta poderosa para los desarrolladores de software, ya que les permiten visualizar el sistema que están construyendo de una manera clara y concisa. Además, estos diagramas pueden ayudar a los desarrolladores a identificar problemas potenciales en el diseño del sistema antes de que se implementen.

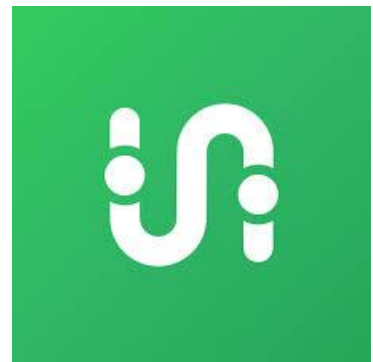
Entre las referencias para este proyecto tenemos:

1. Moovit: una aplicación de transporte público en tiempo real que proporciona información sobre rutas, horarios y tarifas de transporte público. Actualmente está disponible en más de 3.000 ciudades de todo el mundo. [4]



2. Citymapper: otra aplicación de transporte público que proporciona información en tiempo real sobre rutas, horarios y tarifas, además de ofrecer opciones de viaje en bicicleta y a pie. Actualmente está disponible en más de 40 ciudades de todo el mundo.

3. Transit App: una aplicación de transporte público en tiempo real que proporciona información sobre rutas, horarios y tarifas, así como alertas de tráfico y retrasos en tiempo real. Actualmente está disponible en más de 200 ciudades de todo el mundo.



4. Google Maps: una aplicación de mapas y navegación que proporciona información sobre rutas y horarios de transporte público, así como opciones de navegación en coche, bicicleta y a pie. Actualmente está disponible en todo el mundo.[5]



5. Moovit México: una aplicación de transporte público que proporciona información en tiempo real sobre rutas, horarios y tarifas en México. Es similar a Moovit, pero con información específica para el país.

Así mismo, tenemos el caso más importante para nosotros, que es la recarga a la tarjeta MI (movilidad integrada) de la Ciudad de México a través de la aplicación de MercadoPago (mediante saldo de la misma aplicación, tarjetas de crédito y débito e incluso cuotas de MercadoPago), la cual te permite ingresar a diversos transportes (metro, Metrobús, trolebús, cablebus, etc).[3]

Mediante tecnología NFC, realiza las recargas de la siguiente manera:



Ingresa a la app y selecciona Recargar Transporte en la sección Pagar.



Acércala tu Tarjeta MI a la parte de atrás de tu teléfono, cerca de la cámara.



Elige el monto a recargar y el método de pago de tu preferencia.

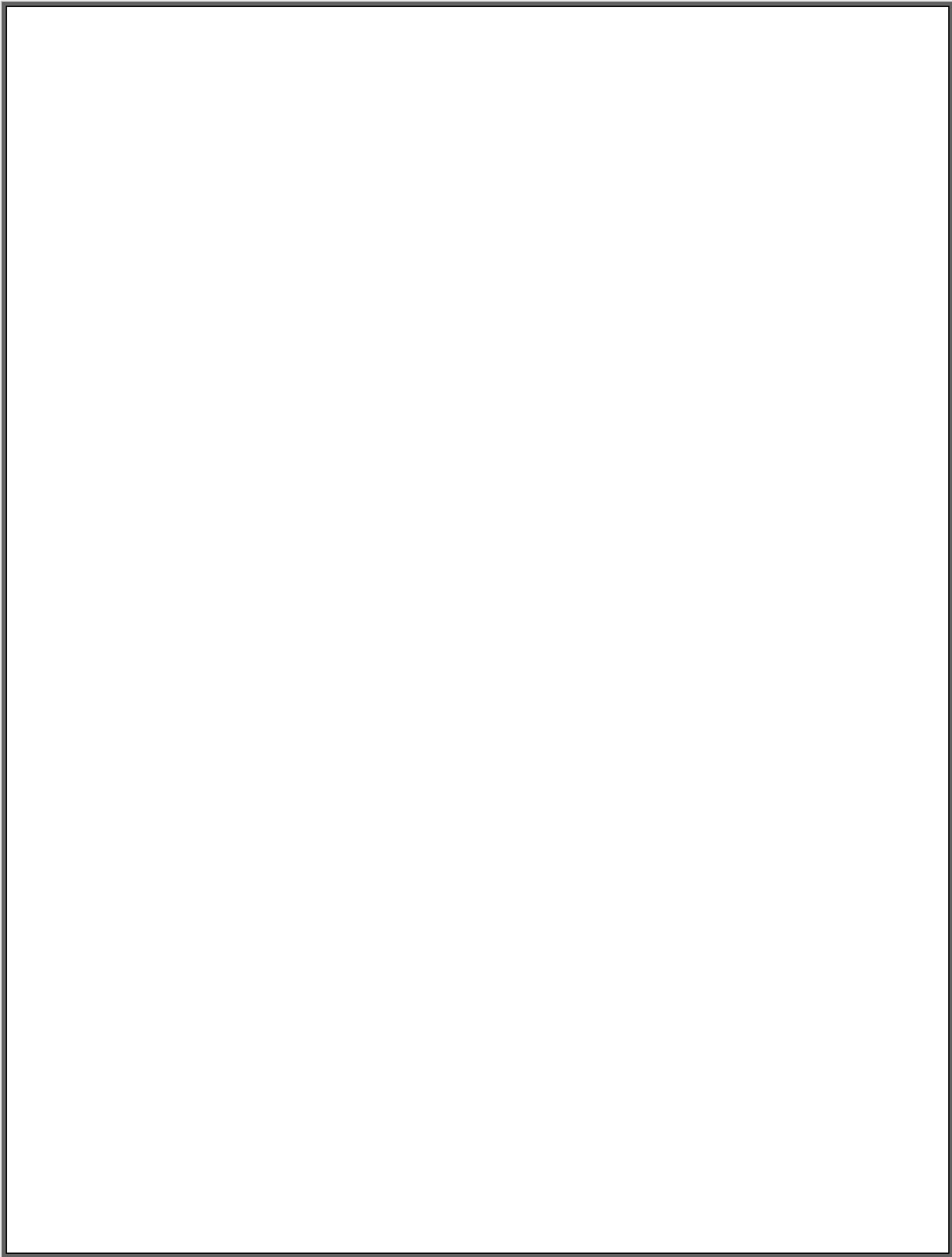


Vuelve a acercar tu tarjeta a la parte de atrás de tu teléfono para acreditar tu recarga y ¡listo!

Requerimientos

- Mejorar la experiencia del usuario cuando ingrese a una estación de la Red Urbana de Transporte articulado: si bien es cierto que se requiere de una nueva tarjeta para el uso de la aplicación RutaMOBILE, es un costo accesible para la comodidad que representara al usuario en el futuro, ya que de cualquier modo es necesario tener una tarjeta que nos permita ingresar a las estaciones.
- Agilizar el ascenso al transporte publico: evitaremos las filas que se forman para recargar las tarjetas que permiten a los usuarios abordar las unidades, ya que desde su hogar podrán tener el saldo disponible para entrar.

- Evitar aglomeraciones en estaciones concurridas: estaciones como la CAPU, BUAP, Paseo Bravo, entre otras, manejan un importante flujo de personas y en estos tiempos de reciente salida de una pandemia, es importante recalcar en que pueden seguir siendo puntos de infección para los usuarios.
- Modernización: siguiendo el ejemplo de grandes ciudades, buscamos el avance tecnológico de la mano con la practicidad y la comodidad, puesto que esto favorece la eficacia del transporte público.[3]
- Manejo más transparente del dinero generado: se podría ofrecer a los usuarios la posibilidad de conocer de manera más detallada el uso que se da al dinero generado por el transporte público. Esto podría hacerse mediante la publicación de informes y estadísticas en línea, que muestren el destino de los recursos y las inversiones realizadas en el transporte público.[4]
- Promoción del uso de la aplicación con diversas estrategias: Como sabemos, las personas mayores tienen un descuento cuando se trata de ingresar a las estaciones, por ello creemos que podemos ayudarlos con RutaMOBILE, ya que permitiría al usuario mayor identificarse como tal y hacer que sus recargas sean de solo el saldo necesario para acceder al transporte público. [5]



Análisis

Mediante el uso de diagramas UML[1] pretendemos mostrar el funcionamiento y objetivo de RutaMOBILE, estos nos darán un panorama más amplio, pero de fácil entendimiento, ya que para ello están diseñados, mostrar los avances del proyecto mediante dichos mapas es una forma eficaz y eficiente de comunicación, especialmente los diagramas de uso y de componentes.

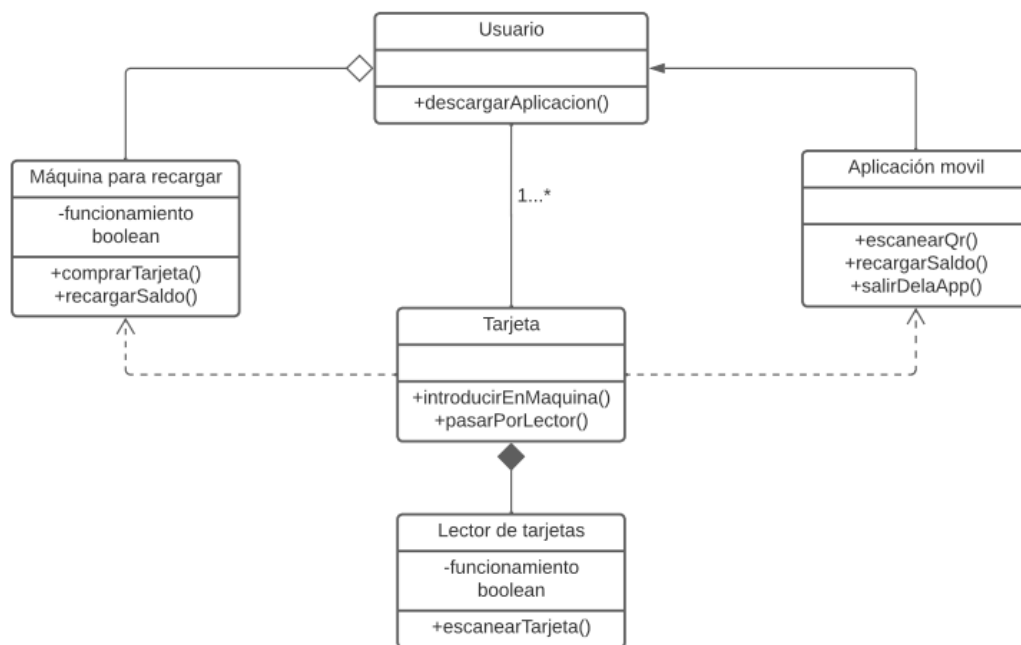


Ilustración 1 Diagrama de clases

En el anterior mapa se puede observar que la operación que debe realizar el usuario sería descargar la aplicación como primera instancia para posteriormente comprar una tarjeta en la máquina de la estación, deberá escanear el código QR único para cada tarjeta que le permitirá recargar saldo mediante

la aplicación, sin embargo, podrá seguir utilizando las máquinas convencionales para agregar dinero a su tarjeta.

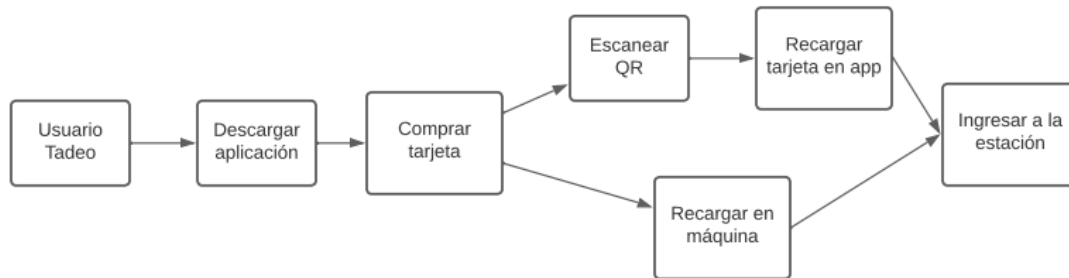


Ilustración 2 Diagrama de objetos

Un usuario común de la red urbana de transporte articulado inicialmente descargará la aplicación y comprará una tarjeta con la que tendrá dos opciones:

1. Escanear su código QR y realizar la recarga desde la aplicación mediante transferencia o agregando una tarjeta de crédito/debito.
2. Insertar la tarjeta en la máquina, agregar la cantidad de dinero deseado y realizar la recarga.

Una vez la tarjeta cuente con el saldo suficiente estará lista para que el usuario pueda abordar la siguiente unidad del RUTA.

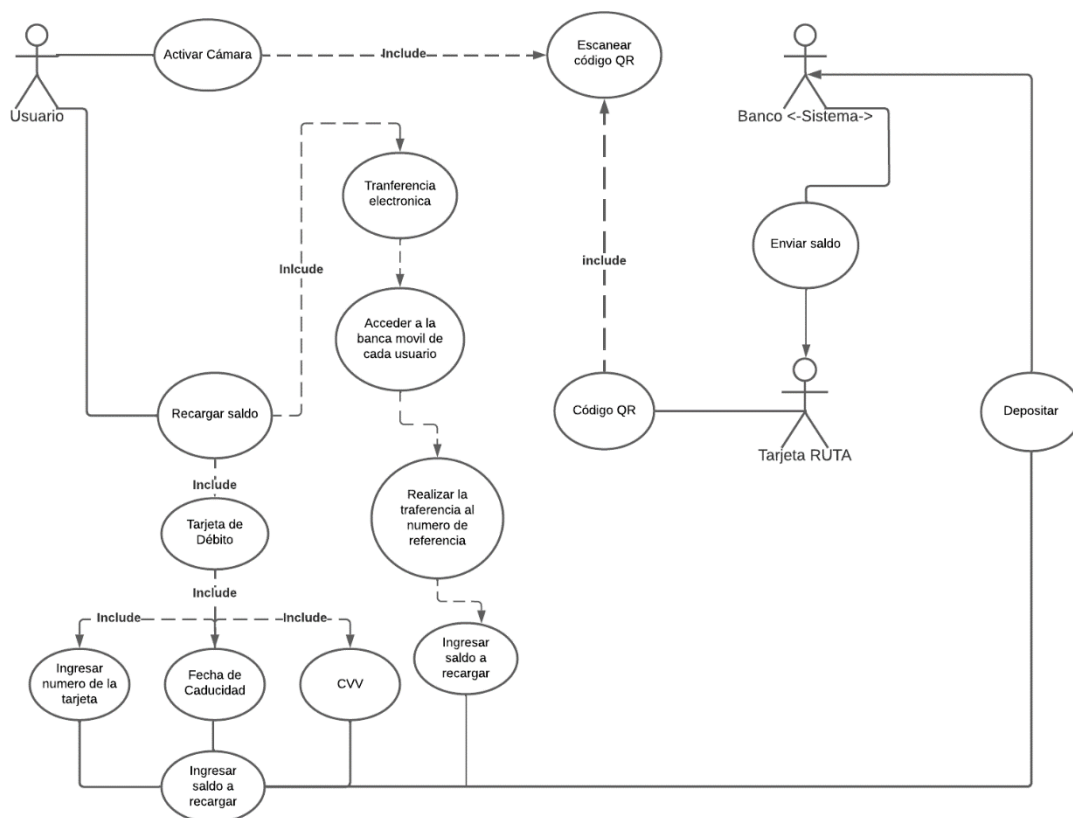


Ilustración 3 Diagrama de casos de uso

El diagrama anterior pertenece a la forma en que funciona el proyecto con respecto a los involucrados en el proceso, podemos observar que el usuario realizara acciones tales como Activar su cámara para proseguir escaneando el QR de la tarjeta.

La acción de Recargar Saldo conlleva a una entidad importante que sería el Banco, a donde llega el dinero y que esta conectada a la entidad de la Tarjeta RutaMOBILE.

Consideramos los siguientes requisitos funcionales:

1. Capacidad para permitir a los usuarios recargar sus tarjetas de transporte público.

2. Facilidad para localizar y seleccionar la estación de transporte público correspondiente.
3. Una interfaz de usuario amigable y fácil de usar que permita a los usuarios navegar por la aplicación de manera intuitiva y rápida.
4. Un proceso de pago seguro y eficiente que permita a los usuarios recargar sus tarjetas de transporte con rapidez.
5. Un sistema de notificaciones que permita a los usuarios recibir información actualizada sobre el estado de su recarga y saldo disponible.

Además de los requisitos funcionales, también es importante considerar los requisitos no funcionales, como el tiempo de respuesta y la eficiencia de la aplicación para garantizar una experiencia satisfactoria para el usuario. Por lo tanto, algunos de los requisitos no funcionales que podrían ser relevantes para RutaMOBILE podrían incluir:

1. Una aplicación rápida y de fácil acceso que permita una experiencia de usuario sin interrupciones.
2. Un diseño responsivo que se adapte a diferentes dispositivos móviles y sistemas operativos.
3. La compatibilidad con diferentes formas de pago (por ejemplo, tarjetas de crédito y débito, sistemas de pago móvil).
4. La protección de la privacidad y la seguridad de los datos personales de los usuarios.

5. La disponibilidad del servicio en línea 24/7, para permitir a los usuarios realizar recargas de tarjetas en cualquier momento y lugar.

De esta manera, pretendemos alcanzar los objetivos planteados al inicio de este proyecto, que velan por el avance tecnológico y la comodidad de los usuarios, pretendemos utilizar las herramientas existentes para presentar un proyecto factible y asegurarnos de continuar por el camino de la innovación.

Uso del Lenguaje de programación

Este lenguaje de programación se utilizó bajo conocimientos previos, tanto el lenguaje y el programa, siendo el más eficiente para realizar la app.

1.1. Lenguaje de programación

Java es un lenguaje de programación ampliamente utilizado en el lenguaje de aplicaciones para Android Studio, el entorno de desarrollo integrado (IDE) oficial de Android, proporciona un entorno completo para programar en Java y construir aplicaciones de alta calidad, alguno de los detalles mas importantes sobre la elección de Java en Android Studio, son las siguientes:

- a) Popularidad y soporte: Java es uno de los lenguajes de programación mas populares y ampliamente adoptados en el mundo y ofrece una amplia variedad de recursos.
- b) Orientación a Objetos: Java es un lenguaje de programación orientado a objetos (OOP), esto significa que se basa en la organización de la lógica de programación en objetos, que encapsulan datos y comportamientos. Esta característica permite una estructura modular y reutilizable, lo que facilita el desarrollo y el mantenimiento de la aplicación.
- c) Sintaxis Limpia y Legible: Java se caracteriza por su sintaxis clara y legible, lo que facilita la comprensión y la depuración del código.
- d) Rendimiento y Portabilidad: Tiene un buen rendimiento y su capacidad para ejecutarse en

diferentes plataformas y sistemas operativos. Esto permite que las aplicaciones desarrolladas en Java para Android Studio sean altamente portátiles y compatibles con una amplia gama de dispositivos.

- e) Amplia Biblioteca Estándar: Cuenta con una extensa biblioteca estándar, que proporciona una gran cantidad de clases y funciones predefinidas.
- f) Manejo de Memoria y Garbage Collection: Utiliza un sistema de gestión automática de memoria, conocido como Garbage Collection. Significa que los desarrolladores no tienen que preocuparse por liberar manualmente la memoria utilizada por los objetos, lo que simplifica el desarrollo y reduce la posibilidad de errores de memoria.
- g) Seguridad: Java ofrece características de seguridad incorporadas, como la gestión de excepciones, que ayuda a evitar vulnerabilidades y a manejar errores de manera controlada.
- h) Compatibilidad con librerías y Frameworks: Java en Android Studio es compatible con una amplia gama de librerías y frameworks de terceros, lo que permite aprovechar el poder de estas herramientas para agilizar el desarrollo y agregar funcionalidades.

1.2. Frameworks

Se utilizó Firebase, que es una plataforma de desarrollo de aplicaciones móviles de Google que proporciona una amplia gama de servicios, simplifica la implementación el uso de base de datos en tiempo

real y almacenamiento en la nube en nuestra aplicación.

También ofrece servicios para pruebas de aplicaciones, indexación y búsqueda, alojamiento web, funciones en la nube (para ejecutar código en la nube), entre otros. La plataforma es altamente escalable, fácil de usar y ofrece una integración fluida con otras herramientas y servicios de Google.

a) Base de Datos en Tiempo Real: Firebase Realtime Database es una base de datos en la nube en tiempo real que permite a los desarrolladores almacenar y sincronizar datos en tiempo real entre clientes y servidores. Es una base de datos NoSQL basada en documentos que facilita la gestión y actualización de datos en tiempo real en aplicaciones colaborativas, como chats, juegos en tiempo real, etc.

b) Almacenamiento en la Nube: Firebase Storage es un servicio de almacenamiento en la nube que permite a los desarrolladores cargar y descargar archivos, como imágenes, videos y otros tipos de contenido, de forma sencilla y eficiente. Proporciona una forma segura de almacenar archivos y ofrece opciones de control de acceso para gestionar quién puede acceder a los archivos.

c) Cloud Firestore: Cloud Firestore es una base de datos en la nube flexible y escalable que permite a los desarrolladores almacenar, sincronizar y consultar datos estructurados en tiempo real. Ofrece una mayor escalabilidad y flexibilidad en

comparación con Firebase Realtime Database, ya que admite consultas más complejas, transacciones y permite un mejor modelado de datos.

d) Analytics: Firebase Analytics es una herramienta de análisis de aplicaciones móviles que proporciona información detallada sobre el rendimiento y el comportamiento de los usuarios en la aplicación. Permite realizar un seguimiento de eventos personalizados, establecer conversiones y obtener información valiosa sobre el rendimiento de la aplicación y el comportamiento de los usuarios para mejorar la toma de decisiones en el desarrollo y la optimización de la aplicación.

e) Crashlytics: Crashlytics es una herramienta de informes de errores y seguimiento de fallos que ayuda a los desarrolladores a identificar y solucionar problemas y bloqueos en la aplicación. Proporciona informes detallados sobre los fallos, incluida la traza de la pila y la información del dispositivo, lo que permite una solución rápida de problemas y una mejora de la estabilidad de la aplicación.

a. Pruebas y Control de Calidad

Se realizaron varias pruebas para comprobar que la aplicación escaneara correctamente el QR y los datos de nuestra tarjeta debito/crédito.

De igual manera se hicieron pruebas de rendimiento para comprobar que tan rápido se escaneaban los datos, y si eran o no correcto lo que se mostraba en pantalla.

Estas pruebas ayudaron a mejorar el código, en base a lo que era más entendible y fácil para nuestros usuarios, ya que la finalidad de esta aplicación es facilitar el uso de la tarjeta del RUTA y evitar complicaciones.

1.3. Respaldo de Datos y Recuperación

El manejo en el respaldo de datos y la recuperación en caso de fallos en la aplicación están protegidos, ya que se uso Firebase como base de datos en tiempo real y gracias a su almacenamiento en la nube proporciona una forma sencilla y eficiente de recuperar datos y de acceder a ellos de manera segura para el usuario.

1.4. Seguridad de la Aplicación

La aplicación es segura, todos los datos personales que llegasen a ser ocupados por nuestra aplicación están almacenados de manera segura, ya que para acceder a la base

de datos únicamente los desarrolladores tienen el control de acceso para gestionar los datos, de igual manera el lenguaje de programación Java nos ofrece bastante seguridad que se implementó en la aplicación.

- a) Sandboxing: Android utiliza un modelo de seguridad basado en sandboxing, lo que significa que cada aplicación se ejecuta en un entorno aislado y no puede acceder directamente a los recursos de otras aplicaciones sin permiso explícito. Esto ayuda a prevenir la interferencia y el acceso no autorizado a datos.
- b) Permisos de la Aplicación: Android permite a los usuarios otorgar o revocar permisos específicos a las aplicaciones. Los permisos deben ser solicitados y aprobados por los usuarios durante la instalación o el primer uso de la aplicación. Esto garantiza que las aplicaciones solo tengan acceso a los recursos y datos para los que se les ha otorgado permiso explícitamente.
- c) Actualizaciones de Seguridad: Google, el desarrollador de Android, regularmente lanza actualizaciones de seguridad para abordar vulnerabilidades conocidas y mejorar la seguridad del sistema operativo.
- d) Prevención de Ataques: Java en Android Studio incluye medidas de seguridad para prevenir y mitigar diversos tipos de ataques, como ataques de inyección de código,

ataques de fuerza bruta, ataques de intermediarios, entre otros.

b. Prueba de caja blanca

Se realizaron distintas pruebas internas del código de la app para ver qué tan efectiva era, estos fueron los resultados.

1.- Prueba del método onCreate(): Se ejecutó varias veces para analizar si la actividad se realizaba de forma correcta y sin errores, así como establecer correctamente la Actividad de la app

Resultado: Todas las pruebas fueron exitosas, ya que no se presentó ningún problema a la hora de ejecutar dicho método, se realizaba y establecía correctamente la actividad en cada ejecución realizada

2.- Prueba del método onClick() del botón mAlertBtn: Se ejecutó varias veces para analizar y verificar que el cuadro de diálogo con las opciones "Si" y "No" funcionen de forma correcta.

Resultado: La verificación del cuadro de diálogo fue exitosa en cada prueba ya que al hacer click en el botón mAlertBtn este funcionaba sin problema, el "Si" funcionaba de forma correcta ya que cancelaba el cuadro de diálogo y el "No" no cerraba la actividad, por lo tanto, funcionó de forma correcta para cada prueba realizada.

3.- Prueba del método onClick() del botón btnScan: Se realizaron distintas pruebas para verificar que se inicie la

lectura de cualquier código QR por medio de la librería IntentIntegrator.

Resultado: Por medio de distintas pruebas se encontró un error inicial debido a que la lectura del código QR no se iniciaba correctamente al hacer clic en el botón btnScan, se corrigió la línea de código correspondiente, dicha falla era un error de sintaxis, nada significativo. Se verificó de forma correcta que los códigos QR válidos se escanean correctamente y que se cancelan correctamente cuando es solicitado, por lo tanto, al depurar dicha falla localizada, todo funcionó correctamente en cada prueba realizada.

4.- Prueba del método `onActivityResult()`: Este método se ejecutó varias veces para corroborar que se reciba correctamente el resultado de la actividad que se inició previamente para escanear el código QR.

Resultado: Se corrigió una falla que existía debido al error en la prueba anterior sobre la lectura del código QR, una vez solucionada dicha falla, se recibe correctamente el resultado de la actividad iniciada previamente para escanear cualquier código QR, al cancelar la lectura, el mensaje "Lectura cancelada" se mostró correctamente en cada prueba realizada y se muestra correctamente el contenido del código de barras escaneado.

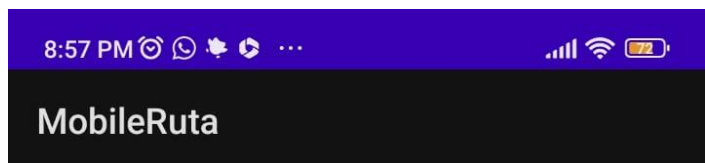
5.- Prueba de la clase `AlertDialog`: Se debe verificar por medio de esta clase que se cree un cuadro de diálogo correctamente con las opciones "Si" y "No", y en este caso en cada prueba realizada no existieron errores, ya

que se creó correctamente el cuadro de diálogo, al dar clic en SI, se cancelaba el cuadro de diálogo y al hacer clic en No la actividad se cerraba correctamente.

6.- Análisis de valores límite: Se analizaron los límites de entrada de datos a la App, ya que los errores a nivel de programación son más propensos a ocurrir en los límites de los valores de entrada, se identificaron los campos de entrada de datos (texto, escaneo) los cuales funcionaron de forma correcta definiendo los valores límite mínimo, mínimo +1, máximo y máximo -1 para cada campo de entrada, los cuales funcionaron de forma correcta en cada prueba realizada dentro del rango de entrada válido para cada caso

7.- Flujo de Control: Se realizaron las pruebas pertinentes para corroborar que el flujo de la app se comportaba de manera prevista, se aseguró que el usuario puede navegar a través de la App de forma correcta y rápida, cumpliendo todas las condiciones y restricciones de flujo correspondientes.

De esta forma, se comprobó que la aplicación a nivel interno funciona de forma correcta y efectiva para que su funcionamiento sea optimo a la hora de navegar en ella, puesto que. Se corrigieron los errores existentes y se le dio atención a las mejoras que se podían realizar a lo largo de cada prueba realizada.



Saldo disponible

AÑADIR

ESCANEAR

Monto a recargar

RECARGAR

AÑADIR TARJETA



En la siguiente prueba de caja negra, observamos que se inicia abriendo la aplicación y continúa agregando la tarjeta al sistema mediante el escaneado de su código QR único.



8:58 PM



MobileRuta



800 800 7
@PueblaR
@PueblaR

Continuando observamos como la tarjeta fue escaneada y agregada correctamente, siendo su saldo disponible mostrado en pantalla

Saldo disponible

01 02 023056165688MRA19IR Saldo \$24.50

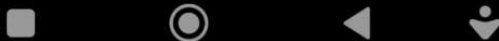
AÑADIR

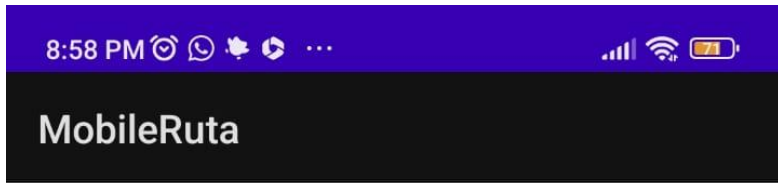
ESCANEAR

Monto a recargar

RECARGAR

AÑADIR TARJETA





La siguiente ventana nos permite agregar un método de pago para recargar con facilidad la tarjeta.



Puedes escanear tu Tarjeta de Débito/Crédito de manera manual o escaneando la parte de enfrente de tu Tarjeta.

Número de Tarjeta

Fecha de Vencimiento

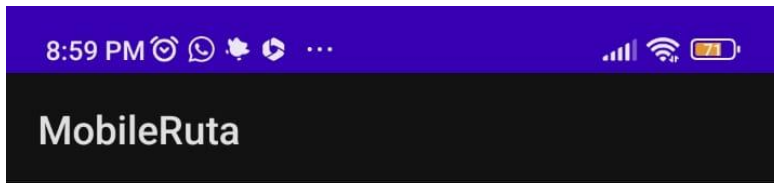
CVV

Nombre del titular

ESCANEAR TARJETA

REGISTRAR TARJETA





Finaliza con la recarga de la tarjeta luego de ingresar los datos y proceder a su registro.



Puedes escanear tu Tarjeta de Débito/Crédito de manera manual o escaneando la parte de enfrente de tu Tarjeta.

Número de Tarjeta

.... 0438

Fecha de Vencimiento

5/2026

CVV

888

Nombre del titular

Berenice Juárez González



Creado exitosamente

ESCANEAR TARJETA

REGISTRAR TARJETA



Conclusiones

El proyecto que realizamos concluyó de manera exitosa. Nuestro objetivo principal era tomar acción para mejorar la eficiencia del transporte público RUTA (Red Urbana de Transporte Articulado), agilizando el ascenso al transporte y con esto mejorando la experiencia del usuario. Durante el desarrollo del proyecto se presentaron distintos desafíos, como la falta de recursos y la necesidad de coordinar diferentes departamentos para asegurar que se implementara de la mejor manera posible.

A pesar de estos obstáculos, se logró desarrollar un sistema funcional y eficaz que permite agilizar y volver más cómodo para el usuario el manejo de su tarjeta del transporte público y el uso de este.

Durante el desarrollo del proyecto aprendimos sobre la importancia de la planificación y la comunicación efectiva y su impacto en la realización de proyectos exitosos.

Agradecemos a todos los miembros del equipo por su trabajo para hacer esto posible y a los colaboradores externos que nos brindaron su tiempo y su apoyo.

Esperamos que este proyecto continúe siendo una herramienta útil para los usuarios del transporte público y que más personas puedan disfrutar de la comodidad que aporta.

En conclusión, la ingeniería de software desempeña un papel fundamental en nuestra sociedad tecnológica.

Su importancia radica en su capacidad para crear y mantener software confiable, escalable y centrado en el usuario. A través de procesos eficientes de desarrollo, fomento de la colaboración y comunicación, garantía de calidad, consideración de la escalabilidad y el mantenimiento, y mejora de la experiencia del usuario, la ingeniería de software impulsa la innovación, mejora la productividad y transforma industrias enteras.

La ingeniería de software permite que los sistemas y aplicaciones que utilizamos en nuestra vida diaria sean más confiables, eficientes y adaptables a medida que nuestras necesidades evolucionan. Además, tiene un impacto significativo en áreas críticas como la atención médica, el transporte, la comunicación y el entretenimiento, mejorando la calidad de vida de las personas y facilitando el progreso de la sociedad en su conjunto.

Bibliografía

«Pruebas unitarias de software: definición, características y ventajas».

<https://www.testingit.com.mx/blog/pruebas-unitarias-de-software>

J. Ramos, «Los diferentes tipos de Pruebas de software», Programación y más.

<https://programacionymas.com/blog/tipos-de-testing-en-desarrollo-de-software>

Atlassian, «The different types of testing in software», Atlassian. <https://www.atlassian.com/continuous-delivery/software-testing/types-of-software-testing>

A. Green, «Metodología Cascada: Beneficios, Uso y Ejemplos Gratis», 2 de noviembre de 2021.

<https://gitmind.com/es/metodologia-cascada.html>

Asana, «Qué es la metodología waterfall y cuándo utilizarla •», Asana.

<https://asana.com/es/resources/waterfall-project-management-methodology>

Rosa Carolina Cortes Galvis, «Modelo de desarrollo de Software (CASCADA)- Ejemplo de aplicación», YouTube. 21 de abril de 2017. [En línea]. Disponible en:

<https://www.youtube.com/watch?v=V05gm73c5ds>

TallerData, «Cronograma de Actividades en Excel», YouTube. 6 de noviembre de 2020. [En línea]. Disponible en: <https://www.youtube.com/watch?v=MdxluZA7DYc>

M. J. C. y B. J. A., Ingeniería del Software: Un enfoque basado en procesos (Spanish Edition). Universidad de Los Andes (980-11), 2021.

[1] M. J. C. y B. J. A., Ingeniería del Software: Un enfoque basado en procesos (Spanish Edition). Universidad de Los Andes (980-11), 2021.

[2] T. Másters, «Te mostramos cómo hacer un marco teórico correctamente», Tesis y Másters Argentina, 18 de enero de 2023. <https://tesisymasters.com.ar/como-hacer-un-marco-teorico/>

[3] M.-D. C. V. de Todo, «Tarjetami». <https://www.mercadopago.com.mx/c/tarjetami>

[4] M. L. M. Matínez, «Moovit, la aplicación para mejorar movilidad de usuarios del transporte público». <https://puebla.gob.mx/index.php/noticias/item/3204-moovit-la-aplicacion-para-mejorar-movilidad-de-usuarios-del-transporte-publico>

[6] «Guía de uso de Metro en lectura fácil», Metro de Madrid. <https://www.metromadrid.es/es/guia-de-uso-de-metro-en-lectura-facil>

[5] S. M., «Cómo funciona el sistema de navegación de Google Maps», abc, 31 de julio de 2019. https://www.abc.es/motor/reportajes/abci-como-funciona-sistema-navegacion-google-maps-201907311604_noticia.html

[1] M. J. C. y B. J. A., Ingeniería del Software: Un enfoque basado en procesos (Spanish Edition). Universidad de Los Andes (980-11), 2021.

S. Siriwardhana, «La Guía Fácil de los Diagramas de Clases UML | Tutorial de Diagramas de Clases», Blog de Creately, sep. 2022, [En línea]. Disponible en:

<https://creately.com/blog/es/diagramas/tutorial-de-diagrama-de-clases/>

[3] M.-D. C. V. de Todo, «Tarjetami».

<https://www.mercadopago.com.mx/c/tarjetami>

[4] D. Morales, «Las tres líneas de RUTA son una carga para las finanzas de Puebla: les han transferido un subsidio de 858 millones de pesos para “mejorar” su operación», 5 de septiembre de 2019.

<https://www.periodicocentral.mx/2019/gobierno/item/10688-las-tres-lineas-de-ruta-son-una-carga-para-las-finanzas-de-puebla-les-han-transferido-un-subsidio-de-858-millones-de-pesos-para-mejorar-su-operacion>

[5] Explora, «¿Eres estudiantes y no tienes tarjeta del RUTA? Aquí te decimos cómo obtenerla», Periódico El Popular, 8 de febrero de 2019. [En línea]. Disponible en: <https://elpopular.mx/secciones/puebla/2019/02/08/eres-estudiantes-y-no-tienes-tarjeta-del-ruta-aqui-te-decimos-como-obtenerla>

Anexo

Codificación

```
//AlertDialog y IntentIntegrator.
package com.example.mobileruta;
//Estas son las importaciones de las clases necesarias para la
aplicacion, incluyendo
//AlertDialog, AppCompatActivity, IntentIntegrator.
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;

import android.content.DialogInterface;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import com.google.zxing.integration.android.IntentIntegrator;
import com.google.zxing.integration.android.IntentResult;

//Se declaran las variables para los botones y el campo de texto.
public class MainActivity extends AppCompatActivity { //Se declara la
clase MainActivity

    Button btnScan; //boton que escanea el QR
    EditText txtResultado; //linea de texto donde muestra los datos que
tiene el QR
    private Button mAlertBtn; //Boton de recargar

    @Override
    protected void onCreate(Bundle savedInstanceState) { //Este es el
método onCreate() que se llama cuando se crea la actividad.
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main); //Se establece el diseño
de la actividad mediante setContentView().

        //Se inicializan los botones y el campo de texto.
        mAlertBtn = (Button) findViewById(R.id.BottonAlert);
        btnScan=findViewById(R.id.btnScan);
        txtResultado=findViewById(R.id.txtResultado);

        //Se establece el OnClickListener para el botón mAlertBtn. Cuando
se hace clic en este botón,
        //se muestra un diálogo de alerta que pregunta si desea continuar
en la aplicación.
        mAlertBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                AlertDialog.Builder alerta = new
AlertDialog.Builder(MainActivity.this); //Se crea el objeto
```



```

AlertDialog.Builder con un mensaje y dos botones, "Si" y "No".
        alerta.setMessage("Recarga lista, el monto ingresado se
        vera reflegado cuando vuelva a escanear, ¿Desea continuar en la
        aplicacion?") //Establece el mensaje que se muestra en el cuadro de
        diálogo.

        .setCancelable(false) //Impide que el cuadro de
        diálogo sea cancelado presionando fuera de él.
        .setPositiveButton("No", new
        DialogInterface.OnClickListener() { //establece un botón negativo con el
        texto "No".

                @Override
                public void onClick(DialogInterface
        dialogInterface, int i) { //Finaliza la actividad actual cuando se
        preciona el botón.

                        finish();
                }
        })
        .setNegativeButton("Si", new
        DialogInterface.OnClickListener() { //establece un botón positivo con el
        texto "Si"

                @Override
                public void onClick(DialogInterface
        dialogInterface, int i) { //Cancela el diálogo cuando se presiona el
        botón.

                        dialogInterface.cancel();
                }
        });

        AlertDialog titulo = alerta.create(); //Se llama al método
        create() para crear el objeto AlertDialog.
        titulo.setTitle("¡Aviso!"); //Se establece un título con
        el método setTitle().
        titulo.show(); //Se muestra el cuadro de diálogo con el
        método show().
    }

    });

    //Se estable el OnClickListener para el botón btnScan. Cuando se
    hace clic en este botón, se inicia
    //la lectura de un código QR utilizando la biblioteca
    IntentIntegrator.
    btnScan.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            //En el método onClick(), se crea una instancia de la clase
            IntentIntegrator, que es una utilidad para iniciar la lactura de códigos
            de barras y QR.
            //Se configura el IntentIntegrator con los formatos de códigos de barras
            que se desean leer, el texto que se muestra como indicación al usuario y
            la cámara que se se utilizará para la lactura, además, se habilita la
            emisión de un pitido y la captura de una imagen del código de barras
            leído.

            IntentIntegrator integrador = new
            IntentIntegrator(MainActivity.this);

            integrador.setDesiredBarcodeFormats(IntentIntegrator.ALL_CODE_TYPES);
            integrador.setPrompt("Lector - CDP");
            integrador.setCameraId(0);

```

```

        integrador.setBeepEnabled(true);
        integrador.setBarcodeImageEnabled(true);
        integrador.initiateScan();
    }
    });
}

//Este método se encarga de recibir el resultado de la
actividad que se inicio previamente para escanear el código.
protected void onActivityResult(int requestCode, int resultCode,
Intent data){//El método onActivityResult() es invocada automáticamente
cuando la actividad anterior finaliza y devuelve un resultado
    IntentResult result =
IntentIntegrator.parseActivityResult(requestCode, resultCode, data);
//crea una instancia de IntentResult al llamar al método
parseActivityResult de la clase IntentIntegrator

//pasando como parámetros el código de solicitud, el resultado y el
objeto Intent devuelto por la actividad de escaneo.

    //Se verifica si la instancia de IntentResult no es nula y si el
contenido es nulo, se muestra el mensaje de "Lectura cancelada" en el
objeto Toast.
    if(result != null){
        if(result.getContents() == null){
            Toast.makeText(this, "lectura cancelada",
Toast.LENGTH_SHORT).show();
        }
        //Si el contenido no es nulo, se muestra el contenido del
código de barras escaneado en otro objeto Toast y se establece como texto
del campo txtResultado.
        else{
            Toast.makeText(this, result.getContents() ,
Toast.LENGTH_SHORT).show();
            txtResultado.setText(result.getContents());
        }
    } else{
        super.onActivityResult(requestCode, resultCode, data);
    }
}
}

```